PROFESSOR KEYBOARD — COMPLETE SOURCE LISTING
ملاحظة: هذا تفريغ نصي لملفات المشروع الموجودة بالحزمة، وليس بديلاً عن بنية المجلدات عند البناء.

==
FILE: .github/workflows/android.yml
==
```
name: Android APK
on:
  workflow_dispatch:
  push:
    branches: [ main, master ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - uses: android-actions/setup-android@v3
      - uses: gradle/actions/setup-gradle@v4
        with:
          gradle-version: '8.9'
      - name: Build debug APK
        run: gradle assembleDebug
      - uses: actions/upload-artifact@v4
        with:
          name: PROFESSOR-KEYBOARD-debug
          path: app/build/outputs/apk/debug/app-debug.apk
```

==
FILE: .gitignore
==
```
.gradle/
.idea/
local.properties
/build/
/app/build/
*.iml
.DS_Store
```

==
FILE: CHANGELOG_AR.md
==
```
# سجل الإصدارات

## 1.0.0 — حزمة المصدر الأولى
- إضافة خدمة لوحة مفاتيح أندرويد أصلية.
- إضافة تخطيط عربي/إنجليزي وأزرار التحكم الأساسية.
- إضافة لوحة رموز وزخارف وإيموجي.
- إضافة لصق من الحافظة وسمات محلية واهتزاز اختياري.
- تثبيت علامة PROFESSOR في رأس لوحة المفاتيح.
```

==
FILE: PROJECT_TREE.txt
==
```
PROFESSOR_KEYBOARD_COMPLETE/
├── .github/workflows/android.yml
├── app/
│   ├── build.gradle.kts
│   ├── proguard-rules.pro
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/com/professor/keyboard/
│       │   ├── ProfessorImeService.kt
│       │   └── SettingsActivity.kt
│       └── res/
│           ├── values/strings.xml
```

```
│           ├── values/styles.xml
│           └── xml/method.xml
├── build.gradle.kts
├── gradle.properties
├── settings.gradle.kts
├── README_AR.md
├── CHANGELOG_AR.md
└── PROJECT_TREE.txt

================================================================================
FILE: README_AR.md
================================================================================
# PROFESSOR KEYBOARD — حزمة المصدر الكاملة

هذا مشروع أندرويد أصلي بلغة Kotlin لخدمة لوحة مفاتيح (IME)، وليس صفحة HTML. العلامة **PROFESSOR** ثابتة داخل الكود وتظهر في رأس لوحة المفاتيح.

## ما يتضمنه الإصدار
- لوحة مفاتيح نظام فعلية يمكن تفعيلها من إعدادات أندرويد.
- تخطيط عربي وإنجليزي والتبديل بينهما.
- أحرف كبيرة/صغيرة للإنجليزية، حذف، مسافة، إدخال، وعلامات ترقيم.
- شريط اقتراحات بسيط مبني على قاموس محلي صغير، بلا اتصال إنترنت.
- لوحة رموز وزخارف نصية، ولوحة إيموجي.
- زر لصق من حافظة النظام، وإعدادات للسمة والاهتزاز.
- إعدادات لتفعيل لوحة المفاتيح وفتح صفحة اختيارها.
- سير عمل GitHub Actions لبناء APK تجريبي.

## البناء
يتطلب Android Studio أو بيئة بناء أندرويد متوافقة، JDK 17، وAndroid SDK Platform 35. افتح المجلد كمشروع Gradle ثم نفّذ `gradle assembleDebug` أو شغّل سير العمل من تبويب Actions في GitHub.

## التفعيل
1. ثبّت APK الناتج.
2. افتح PROFESSOR KEYBOARD.
3. اختر Enable Keyboard ثم فعّلها من إعدادات النظام.
4. اختر Select Keyboard وحدد PROFESSOR KEYBOARD.

## الخصوصية والحدود الحالية
- لا يرسل المشروع ضغطات المفاتيح إلى خادم أو Manifest؛ لا توجد صلاحية إنترنت في AndroidManifest.
- القاموس محلي صغير كبداية، وليس محرك تنبؤ لغوي متقدمًا.
- لا يتضمن هذا الإصدار التعرف الصوتي، أو الكتابة بالسحب، أو GIF عبر الإنترنت، أو مزامنة سحابية، أو محرر خلفيات متحرك. لا يصح وصف هذه الوظائف بأنها مكتملة هنا.
- قد يقيّد الوصول إلى الحافظة حسب إصدار النظام وحالة التطبيق في أندرويد.
- لم يتم بناء APK أو اختباره على جهاز فعلي داخل هذه البيئة؛ يلزم البناء والاختبار قبل التوزيع.

================================================================================
FILE: app/build.gradle.kts
================================================================================
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.professor.keyboard"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.professor.keyboard"
        minSdk = 23
        targetSdk = 35
        versionCode = 1
        versionName = "1.0.0"
    }
    buildTypes {
        release {
            isMinifyEnabled = false
```

```
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-
rules.pro")
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions { jvmTarget = "17" }
}

================================================================================
FILE: app/proguard-rules.pro
================================================================================
# No custom shrinking rules are required for this first complete source release.

================================================================================
FILE: app/src/main/AndroidManifest.xml
================================================================================
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="false"
        android:label="PROFESSOR KEYBOARD"
        android:theme="@style/AppTheme"
        android:supportsRtl="true">
        <activity android:name=".SettingsActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service
            android:name=".ProfessorImeService"
            android:label="PROFESSOR KEYBOARD"
            android:permission="android.permission.BIND_INPUT_METHOD"
            android:exported="true">
            <intent-filter><action android:name="android.view.InputMethod" /></intent-filter>
            <meta-data android:name="android.view.im" android:resource="@xml/method" />
        </service>
    </application>
</manifest>

================================================================================
FILE: app/src/main/java/com/professor/keyboard/ProfessorImeService.kt
================================================================================
package com.professor.keyboard

import android.content.Context
import android.graphics.Color
import android.graphics.Typeface
import android.inputmethodservice.InputMethodService
import android.os.Vibrator
import android.view.Gravity
import android.view.View
import android.view.ViewGroup
import android.view.inputmethod.EditorInfo
import android.widget.Button
import android.widget.HorizontalScrollView
import android.widget.LinearLayout
import android.widget.TextView

class ProfessorImeService : InputMethodService() {
    private companion object { const val BRAND_LABEL = "PROFESSOR" }
    private var language = 0
    private var shifted = false
    private var panel = "letters"
    private lateinit var root: LinearLayout
    private lateinit var keysArea: LinearLayout
    private lateinit var suggestions: LinearLayout

    private val arRows = listOf("ض ص ث ق ف غ ع ه خ ح ج", "ش س ي ب ل ا ت ن م ك ط", "ئ ء ؤ ر لا ى ة و ز ظ د")
    private val enRows = listOf("q w e r t y u i o p", "a s d f g h j k l", "z x c v b n m")
    private val localWords = mapOf(
```

```
        "ar" to listOf("مرحبا", "تمام", "شكرا", "من فضلك", "صباح الخير", "مساء الخير", "أنا", "أنت", "كيف", "جميل"),
        "en" to listOf("hello", "thanks", "please", "good", "morning", "evening", "yes", "no", "great", "welcome")
    )

    override fun onCreateInputView(): View {
        panel = "letters"
        root = LinearLayout(this).apply { orientation = LinearLayout.VERTICAL; setPadding(5, 5, 5, 7) }
        buildKeyboard()
        return root
    }

    private fun prefs() = getSharedPreferences("professor_keyboard", Context.MODE_PRIVATE)
    private fun accent(): Int = when (prefs().getString("theme", "blue")) {
        "purple" -> Color.rgb(85, 48, 130)
        "green" -> Color.rgb(28, 105, 83)
        "black" -> Color.rgb(35, 39, 48)
        else -> Color.rgb(18, 83, 125)
    }
    private fun bg(): Int = Color.rgb(9, 14, 24)

    private fun buildKeyboard() {
        root.removeAllViews()
        root.setBackgroundColor(bg())
        val brand = TextView(this).apply {
            text = "$BRAND_LABEL  •  ULTRA SPEED"
            setTextColor(Color.rgb(75, 190, 255)); textSize = 12f
            typeface = Typeface.DEFAULT_BOLD; gravity = Gravity.CENTER
            setPadding(4, 7, 4, 7)
        }
        root.addView(brand, LinearLayout.LayoutParams(-1, dp(30)))
        val tools = row()
        addKey(tools, "☷", true) { panel = if (panel == "symbols") "letters" else "symbols"; buildKeyboard() }
        addKey(tools, "☺", true) { panel = if (panel == "emoji") "letters" else "emoji"; buildKeyboard() }
        addKey(tools, "▣", true) { pasteClipboard() }
        addKey(tools, if (language == 0) "عربي" else "EN", true) { language = 1 - language; panel = "letters"; buildKeyboard() }
        addKey(tools, "⚙", true) { startActivity(android.content.Intent(this, SettingsActivity::class.java).addFlags(android.content.Intent.FLAG_ACTIVITY_NEW_TASK)) }
        root.addView(tools, LinearLayout.LayoutParams(-1, dp(42)))

        suggestions = row()
        root.addView(suggestions, LinearLayout.LayoutParams(-1, dp(37)))
        updateSuggestions()
        keysArea = LinearLayout(this).apply { orientation = LinearLayout.VERTICAL }
        root.addView(keysArea, LinearLayout.LayoutParams(-1, -2))
        when (panel) {
            "symbols" -> buildSymbols()
            "emoji" -> buildEmoji()
            else -> buildLetters()
        }
    }

    private fun buildLetters() {
        val rows = if (language == 0) arRows else enRows
        rows.forEach { line ->
            val r = row()
            line.split(" ").forEach { key -> addKey(r, if (shifted && language == 1) key.uppercase() else key, false) { commit(if (shifted && language == 1) key.uppercase() else key); if (language == 1) shifted = false } }
            keysArea.addView(r, LinearLayout.LayoutParams(-1, dp(49)))
        }
        val bottom = row()
        addKey(bottom, "⇧", true) { shifted = !shifted; buildKeyboard() }
        addKey(bottom, "?123", true) { panel = "symbols"; buildKeyboard() }
        addKey(bottom, ",", false) { commit(",") }
        addKey(bottom, "English/عربي", true) { language = 1 - language; buildKeyboard() }
        addKey(bottom, ".", false) { commit(".") }
```

```
        addKey(bottom, "⌫", true) { deleteOne() }
        keysArea.addView(bottom, LinearLayout.LayoutParams(-1, dp(49)))
        val last = row()
        addKey(last, "?123", true) { panel = "symbols"; buildKeyboard() }
        addKey(last, "🌐", true) { language = 1 - language; buildKeyboard() }
        addKey(last, "مسافة / SPACE", true, 4f) { commit(" ") }
        addKey(last, "↵", true) { enter() }
        keysArea.addView(last, LinearLayout.LayoutParams(-1, dp(49)))
    }

    private fun buildSymbols() {
        val rows = listOf("1 2 3 4 5 6 7 8 9 0", "@ # $ % & - + ( ) /", "* \" ' : ; ! ? ~ `")
        rows.forEach { line -> val r = row(); line.split(" ").forEach { s -> addKey(r, s, false) {
commit(s) } }; keysArea.addView(r, LinearLayout.LayoutParams(-1, dp(49))) }
        val deco = listOf("★", "♡", "✦", "彡", "『 』", "꒰ ꒱", "ツ", "✓", "∞", "•")
        val r = row(); deco.forEach { s -> addKey(r, s, false) { commit(s) } }; keysArea.addView(r,
LinearLayout.LayoutParams(-1, dp(49)))
        val back = row(); addKey(back, "ABC", true, 2f) { panel = "letters"; buildKeyboard() };
addKey(back, "⌫", true) { deleteOne() }; addKey(back, "مسافة", true, 2f) { commit(" ") };
keysArea.addView(back, LinearLayout.LayoutParams(-1, dp(49)))
    }

    private fun buildEmoji() {
        val emojis = listOf("😀", "😂", "🥰", "😍", "😎", "😭", "😡", "🤔", "👍", "🙏", "❤️",
"🔥", "✨", "🌹", "💯", "🎉", "😊", "😉", "😴", "🤝", "💙", "⭐", "☀️", "🌙")
        emojis.chunked(8).forEach { chunk -> val r = row(); chunk.forEach { e -> addKey(r, e,
false) { commit(e) } }; keysArea.addView(r, LinearLayout.LayoutParams(-1, dp(48))) }
        val back = row(); addKey(back, "ABC", true, 2f) { panel = "letters"; buildKeyboard() };
addKey(back, "⌫", true) { deleteOne() }; keysArea.addView(back, LinearLayout.LayoutParams(-1,
dp(48)))
    }

    private fun row() = LinearLayout(this).apply { orientation = LinearLayout.HORIZONTAL; gravity =
Gravity.CENTER; setPadding(0, 2, 0, 2) }
    private fun addKey(parent: LinearLayout, label: String, special: Boolean, weight: Float = 1f,
action: () -> Unit) {
        val b = Button(this).apply {
            text = label; textSize = if (label.length > 5) 11f else 17f
            isAllCaps = false; setTextColor(Color.WHITE)
            setBackgroundTintList(android.content.res.ColorStateList.valueOf(if (special) accent()
else Color.rgb(34, 42, 56)))
            minHeight = 0; minimumHeight = 0; minWidth = 0; minimumWidth = 0
            setPadding(1, 0, 1, 0)
            setOnClickListener { if (prefs().getBoolean("vibration", false))
(getSystemService(VIBRATOR_SERVICE) as? Vibrator)?.vibrate(12); action() }
        }
        val lp = LinearLayout.LayoutParams(0, -1, weight).apply { setMargins(dp(2), 0, dp(2), 0) }
        parent.addView(b, lp)
    }
    private fun commit(s: String) {
        if (s == " ") applyLocalCorrection()
        currentInputConnection?.commitText(s, 1)
        updateSuggestions()
    }
    private fun applyLocalCorrection() {
        val ic = currentInputConnection ?: return
        val before = ic.getTextBeforeCursor(40, 0)?.toString().orEmpty()
        val word = before.takeLastWhile { !it.isWhitespace() }
        val fixes = if (language == 0) mapOf("ازيك" to "انشاءالله" ,"إزيك" to "شكرا" ,"إن شاء الله" to
"مرحبا" ,"شكرا" to "مرحبا")
                   else mapOf("teh" to "the", "adn" to "and", "thier" to "their", "recieve" to
"receive")
        val fixed = fixes[word.lowercase()] ?: return
        ic.deleteSurroundingText(word.length, 0)
        ic.commitText(fixed, 1)
    }
    private fun deleteOne() { currentInputConnection?.deleteSurroundingText(1, 0) }
    private fun enter() {
        val action = currentInputEditorInfo?.imeOptions?.and(EditorInfo.IME_MASK_ACTION) ?:
EditorInfo.IME_ACTION_NONE
        if (action != EditorInfo.IME_ACTION_NONE && action != EditorInfo.IME_ACTION_UNSPECIFIED)
currentInputConnection?.performEditorAction(action) else currentInputConnection?.commitText("\n",
```

```
1)
    }
    private fun pasteClipboard() {
        val clip = getSystemService(CLIPBOARD_SERVICE) as? android.content.ClipboardManager
        val text = clip?.primaryClip?.getItemAt(0)?.coerceToText(this)?.toString()
        if (!text.isNullOrEmpty()) currentInputConnection?.commitText(text, 1)
    }
    private fun updateSuggestions() {
        if (!::suggestions.isInitialized) return
        suggestions.removeAllViews()
        val lang = if (language == 0) "ar" else "en"
        val before = currentInputConnection?.getTextBeforeCursor(40, 0)?.toString().orEmpty()
        val prefix = before.takeLastWhile { !it.isWhitespace() }
        val words = localWords[lang].orEmpty().filter { prefix.isNotBlank() &&
it.startsWith(prefix, true) && it != prefix }.take(4)
        (if (words.isEmpty()) listOf("$BRAND_LABEL", "${if (language == 0) "عربي" else "English"}")
else words).forEach { word ->
            val t = TextView(this).apply {
                text = word; textSize = 12f; gravity = Gravity.CENTER; setTextColor(Color.WHITE);
setPadding(dp(7), 0, dp(7), 0)
                setOnClickListener {
                    if (prefix.isNotBlank())
currentInputConnection?.deleteSurroundingText(prefix.length, 0)
                    currentInputConnection?.commitText(word, 1)
                    updateSuggestions()
                }
            }
            suggestions.addView(t, LinearLayout.LayoutParams(0, -1, 1f))
        }
    }
    private fun dp(v: Int) = (v * resources.displayMetrics.density).toInt()
}

================================================================================
FILE: app/src/main/java/com/professor/keyboard/SettingsActivity.kt
================================================================================
package com.professor.keyboard

import android.app.Activity
import android.content.Context
import android.content.Intent
import android.graphics.Color
import android.os.Bundle
import android.provider.Settings
import android.view.Gravity
import android.view.inputmethod.InputMethodManager
import android.widget.Button
import android.widget.LinearLayout
import android.widget.Switch
import android.widget.TextView

class SettingsActivity : Activity() {
    private val prefs by lazy { getSharedPreferences("professor_keyboard", Context.MODE_PRIVATE) }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val root = LinearLayout(this).apply { orientation = LinearLayout.VERTICAL; setPadding(22,
28, 22, 22); setBackgroundColor(Color.rgb(8, 13, 23)) }
        fun title(text: String, size: Float = 18f) = TextView(this).apply { this.text = text;
textSize = size; setTextColor(Color.WHITE); gravity = Gravity.CENTER; setPadding(4, 12, 4, 12) }
        root.addView(title("PROFESSOR KEYBOARD", 24f))
        root.addView(title("إعدادات لوحة المفاتيح – الاسم ثابت داخل التطبيق"))
        val enable = Button(this).apply { text = "تفعيل لوحة المفاتيح"; setOnClickListener {
startActivity(Intent(Settings.ACTION_INPUT_METHOD_SETTINGS)) } }
        root.addView(enable)
        val choose = Button(this).apply { text = "اختيار PROFESSOR KEYBOARD"; setOnClickListener {
(getSystemService(INPUT_METHOD_SERVICE) as InputMethodManager).showInputMethodPicker() } }
        root.addView(choose)
        root.addView(title("السمة"))
        listOf("blue" to "أزرق", "purple" to "بنفسجي", "green" to "أخضر", "black" to "داكن").forEach
{ (key, label) ->
            val b = Button(this).apply { text = label; setOnClickListener {
prefs.edit().putString("theme", key).apply() } }; root.addView(b)
        }
```

        val vibration = Switch(this).apply { text = "اهتزاز عند الضغط"; setTextColor(Color.WHITE);
isChecked = prefs.getBoolean("vibration", false); setOnCheckedChangeListener { _, checked ->
prefs.edit().putBoolean("vibration", checked).apply() } }
        root.addView(vibration)
        root.addView(title("14 , ".المعالجة محلية. لا توجد صلاحية إنترنت :الخصوصيةf))
        setContentView(root)
    }
}

==
FILE: app/src/main/res/values/strings.xml
==
<resources><string name="app_name">PROFESSOR KEYBOARD</string></resources>

==
FILE: app/src/main/res/values/styles.xml
==
<resources>
    <style name="AppTheme" parent="android:style/Theme.Material.Light.NoActionBar">
        <item name="android:fontFamily">sans</item>
        <item name="android:colorAccent">#39B8FF</item>
        <item name="android:statusBarColor">#080D18</item>
        <item name="android:navigationBarColor">#080D18</item>
        <item name="android:windowLightStatusBar">false</item>
    </style>
</resources>

==
FILE: app/src/main/res/xml/method.xml
==
<?xml version="1.0" encoding="utf-8"?>
<input-method xmlns:android="http://schemas.android.com/apk/res/android" android:isDefault="false"
android:supportsSwitchingToNextInputMethod="true">
    <subtype android:label="Arabic" android:imeSubtypeLocale="ar" android:imeSubtypeMode="keyboard"
/>
    <subtype android:label="English" android:imeSubtypeLocale="en_US"
android:imeSubtypeMode="keyboard" />
</input-method>

==
FILE: build.gradle.kts
==
plugins {
    id("com.android.application") version "8.7.3" apply false
    id("org.jetbrains.kotlin.android") version "2.0.21" apply false
}

==
FILE: gradle.properties
==
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
org.gradle.parallel=true
android.useAndroidX=true
kotlin.code.style=official

==
FILE: settings.gradle.kts
==
pluginManagement { repositories { google(); mavenCentral(); gradlePluginPortal() } }
dependencyResolutionManagement { repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS);
repositories { google(); mavenCentral() } }
rootProject.name = "PROFESSOR-KEYBOARD"
include(":app")